

The background is a dark gray field filled with a complex pattern of small white dots. Overlaid on this are numerous green and purple dots of varying sizes, some arranged in horizontal lines. There are also thin, wavy green and purple lines and several green-outlined rectangles of different sizes, some of which are tilted or overlapping.

Deployment

Part 4: Automating deployment using CI/CD

In the previous videos...

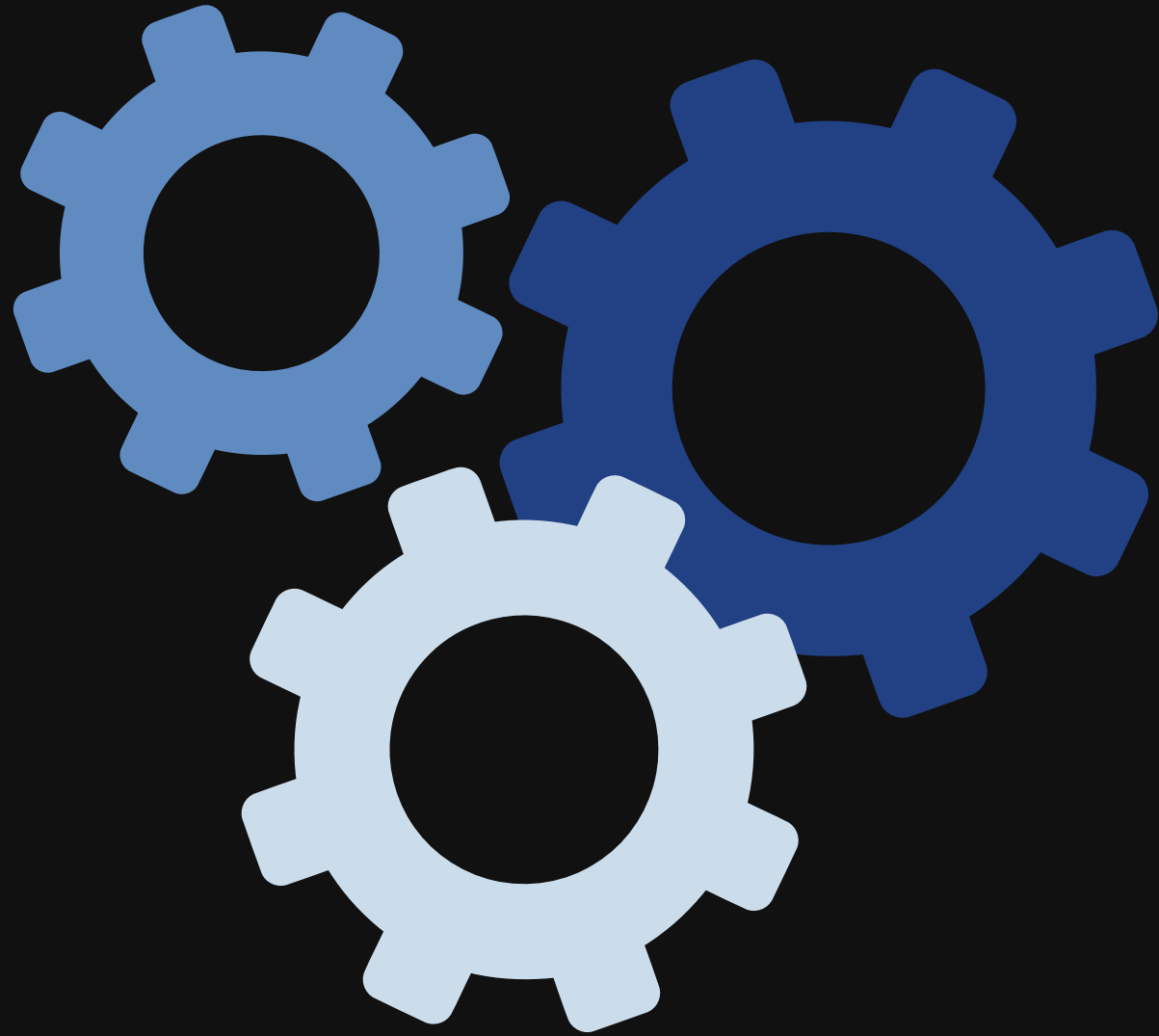


In this video...

- Add GitHub Secrets
- Adjust code for deployment
- Deploy the application



Add GitHub Secrets



Security

* Secrets and variables

Actions

Actions secrets and variables

Secrets and variables allow you to manage reusable configuration data. Secrets are **encrypted** and are used for sensitive data. [Learn more about encrypted secrets](#). Variables are shown as plain text and are used for **non-sensitive** data. [Learn more about variables](#).

Anyone with collaborator access to this repository can use these secrets and variables for actions. They are not passed to workflows that are triggered by a pull request from a fork.

Secrets

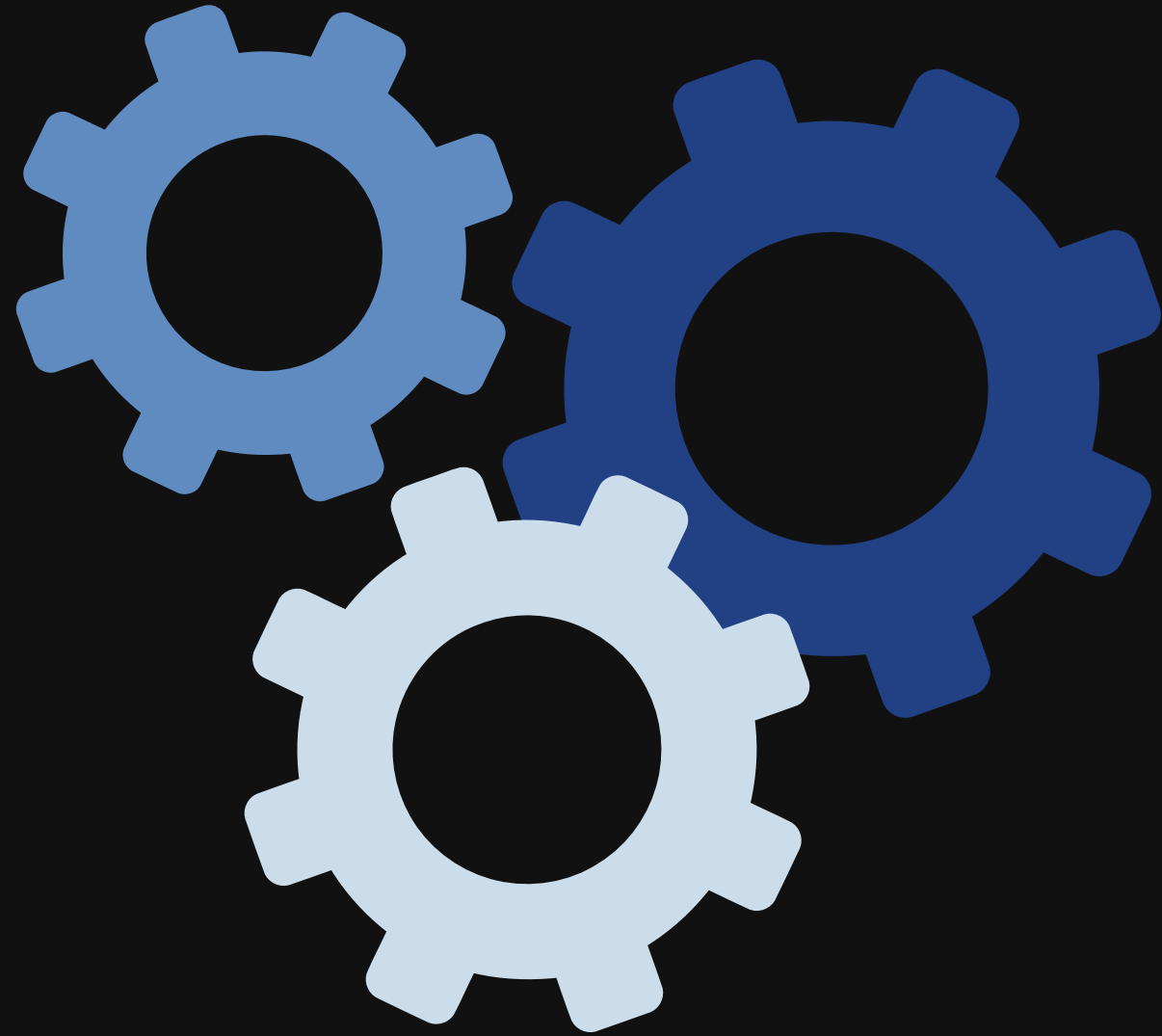
Variables

Repository secrets

New repository secret

Name 	Last updated		
 EC2_HOST	1 hour ago		
 EC2_SSH_KEY	1 hour ago		
 PRODUCTION_API_URL	32 minutes ago		

Adjust Repository Code

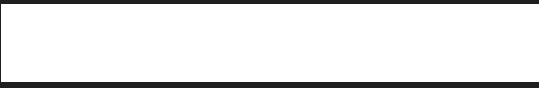


Update Backend Dockerfile

```
1  # Python image
2  FROM python:3.11-slim
3
4  ENV PYTHONUNBUFFERED=1
5
6  # Set working directory
7  WORKDIR /app
8
9  # Command to install Python Packages
10 RUN pip install --no-cache-dir flask flask-cors gunicorn
11
12 # Copy backend source code
13 COPY backend/ ./backend/
14
15 # Expose flask port number
16 EXPOSE 5000
17
18 # Run application
19 CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "2", "backend.app:app"]
```

Update Backend CORS

```
app > backend >  app.py > ...
```

```
1  from backend.calculator import calculate_multiple
2  from flask import Flask, request, jsonify
3  from flask_cors import CORS
4
5  app = Flask(__name__)
6  CORS(app, origins=["http://localhost:5173", "http://"])
7
8  @app.route('/api/calculate', methods=['POST'])
```


Update Frontend Dockerfile

```
1  # Node image
2  FROM node:20 AS builder
3
4  # Set working directory
5  WORKDIR /app
6
7  # Copy package files
8  COPY package*.json ./
9
10 # Install dependencies
11 RUN npm install
12
13 # Copy frontend source code
14 COPY . .
15
16 ARG VITE_API_URL
17 ENV VITE_API_URL=$VITE_API_URL
18
19 RUN npm run build
20
21 FROM nginx:alpine
22 COPY --from=builder /app/dist /usr/share/nginx/html
23
24 # Expose port for Vite
25 EXPOSE 80
26
27 # Start development server
28 CMD ["nginx", "-g", "daemon off;"]
```

Add docker-compose for deployment

```
1  services:
2
3    backend:
4      image: ghcr.io/keanu-ellary/calculator-backend:latest
5      restart: unless-stopped
6      ports:
7        - "5000:5000"
8
9    frontend:
10     image: ghcr.io/keanu-ellary/calculator-frontend:latest
11     restart: unless-stopped
12     ports:
13       - "80:80"
14     depends_on:
15       - backend
```

Add Workflow for deployment

```
name: Production Deployment
```

```
on:
```

```
  push:
```

```
    tags:
```

```
      - 'v*.*.*'
```

```
permissions:
```

```
  contents: read
```

```
  packages: write
```

```
jobs:
```

```
14     production-check:
15         runs-on: ubuntu-latest
16         steps:
17             - name: Checkout code
18               uses: actions/checkout@v4
19
20             - name: Set up node
21               uses: actions/setup-node@v4
22               with:
23                 node-version: "20"
24                 cache: "npm"
25                 cache-dependency-path: app/frontend/package-lock.json
26
27             - name: Install frontend dependencies
28               working-directory: ./app/frontend
29               run: npm ci
30
31             - name: Run frontend tests
32               working-directory: ./app/frontend
33               run: npm run test:unit
34
35             - name: Set up Python
36               uses: actions/setup-python@v5
37               with:
38                 python-version: "3.11.7"
39
40             - name: Install backend dependencies
41               run: |
42                 python -m pip install --upgrade pip
43                 pip install flask flask-cors pytest
44
45             - name: Run backend tests
46               run: pytest
```



```
48   build-and-push:
49     needs: production-check
50     runs-on: ubuntu-latest
51     steps:
52       - name: Checkout code
53         uses: actions/checkout@v4
54
55       - name: Set up Docker Buildx
56         uses: docker/setup-buildx-action@v3
57
58       - name: Log in to GitHub Container Registry
59         uses: docker/login-action@v3
60         with:
61           registry: ghcr.io
62           username: ${ github.actor }
63           password: ${ secrets.GITHUB_TOKEN }
64
65       - name: Build and push frontend
66         uses: docker/build-push-action@v5
67         with:
68           context: ./app/frontend
69           file: ./app/frontend/Dockerfile
70           push: true
71           build-args: |
72             VITE_API_URL=${ secrets.PRODUCTION_API_URL }
73           tags: |
74             ghcr.io/keanu-ellary/calculator-frontend:latest
75             ghcr.io/keanu-ellary/calculator-frontend:${ github.ref_name }
76
77       - name: Build and push backend
78         uses: docker/build-push-action@v5
79         with:
80           context: ./app
81           file: ./app/backend/Dockerfile
82           push: true
83           tags: |
84             ghcr.io/keanu-ellary/calculator-backend:latest
85             ghcr.io/keanu-ellary/calculator-backend:${ github.ref_name }
86
```

87 **deploy:**

88 **needs:** build-and-push

89 **runs-on:** ubuntu-latest

90 **steps:**

91 - **name:** Checkout code

92 **uses:** actions/checkout@v4

94 - **name:** Set up SSH access to EC2

95 **run:** |

96 `mkdir -p ~/.ssh`

97 `echo "${{ secrets.EC2_SSH_KEY }}" > ~/.ssh/id_rsa`

98 `chmod 600 ~/.ssh/id_rsa`

99 `ssh-keyscan -H "${{ secrets.EC2_HOST }}" >> ~/.ssh/known_hosts`

101 - **name:** Copy docker-compose.yml to EC2

102 **run:** |

103 `ssh -i ~/.ssh/id_rsa ubuntu@${{ secrets.EC2_HOST }}"mkdir -p ~/calculator"`

104 `rsync -az -e "ssh -i ~/.ssh/id_rsa" ./app/docker-compose.prod.yml ubuntu@${{ secrets.EC2_HOST }}:~/calculator/docker-compose.yml`

106 - **name:** Pull new images and restart containers

107 **run:** |

108 `ssh -i ~/.ssh/id_rsa ubuntu@${{ secrets.EC2_HOST }}"<< 'EOF'`

109 `cd ~/calculator`

110 `echo "${{ secrets.GITHUB_TOKEN }}" | docker login ghcr.io -u "${{ github.actor }}" --password-stdin`

111 `docker compose pull`

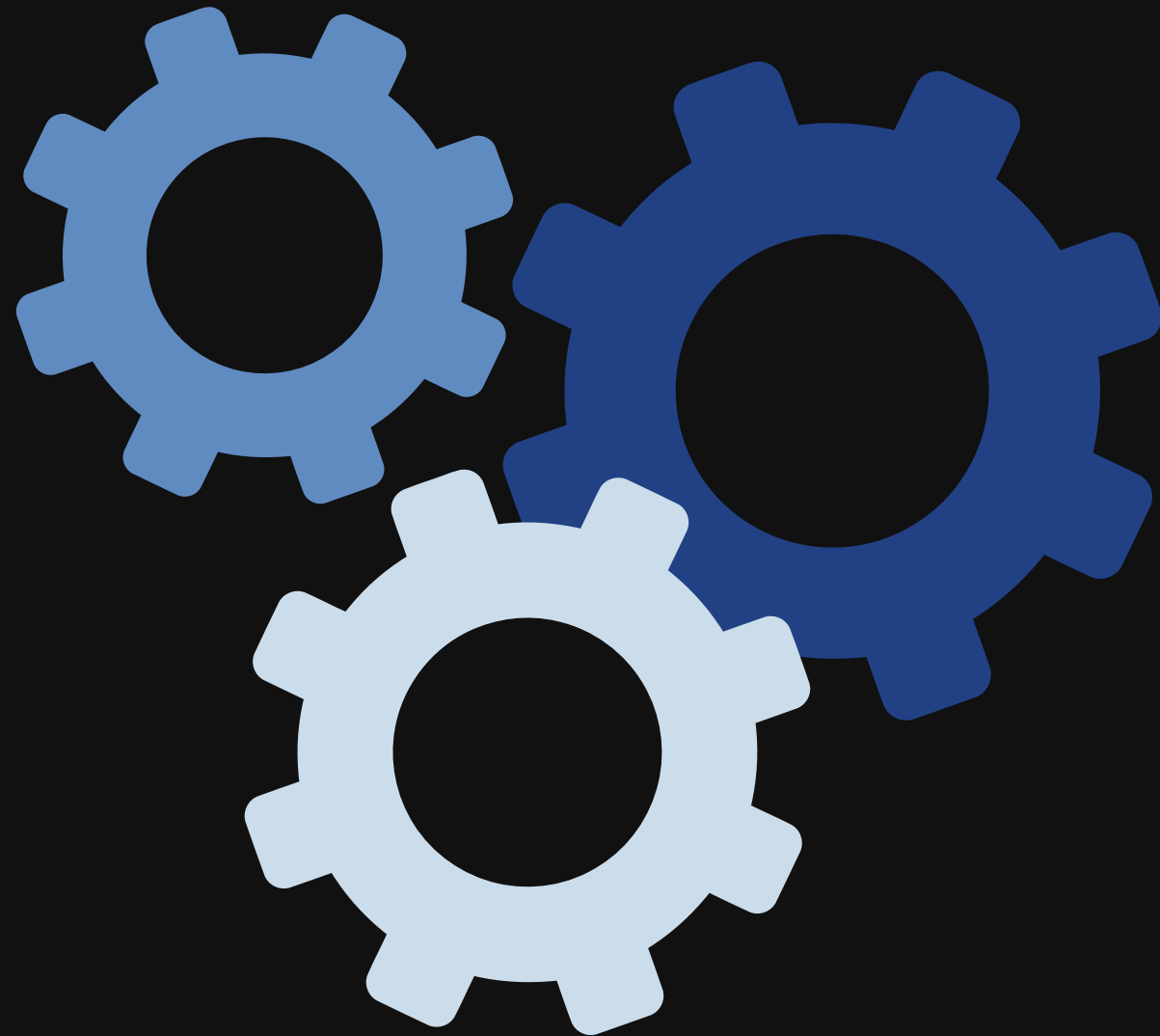
112 `docker compose down || true`

113 `docker compose up -d --remove-orphans`

114 `docker image prune -a -f`

115 `EOF`

Deploy the Application

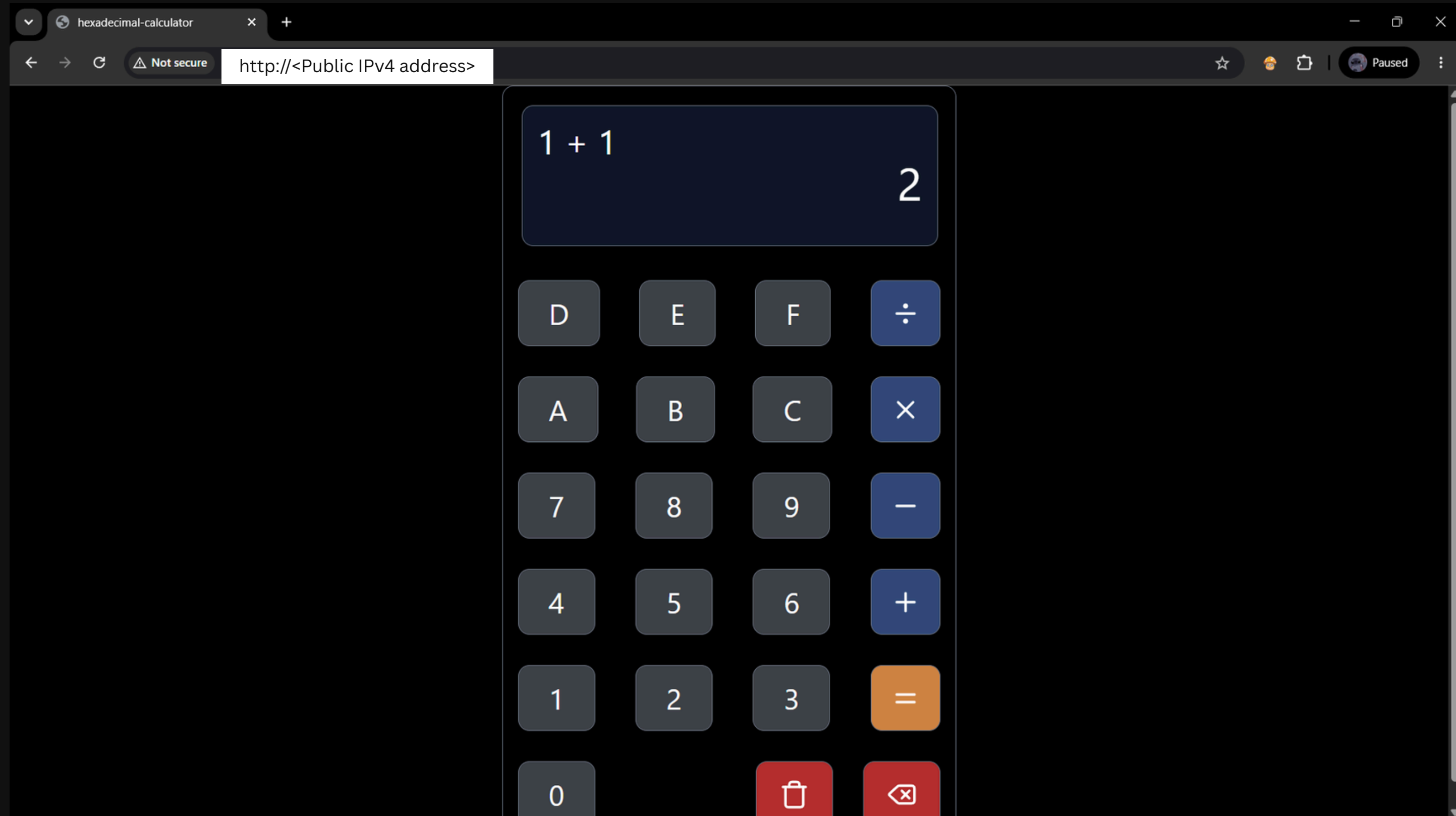


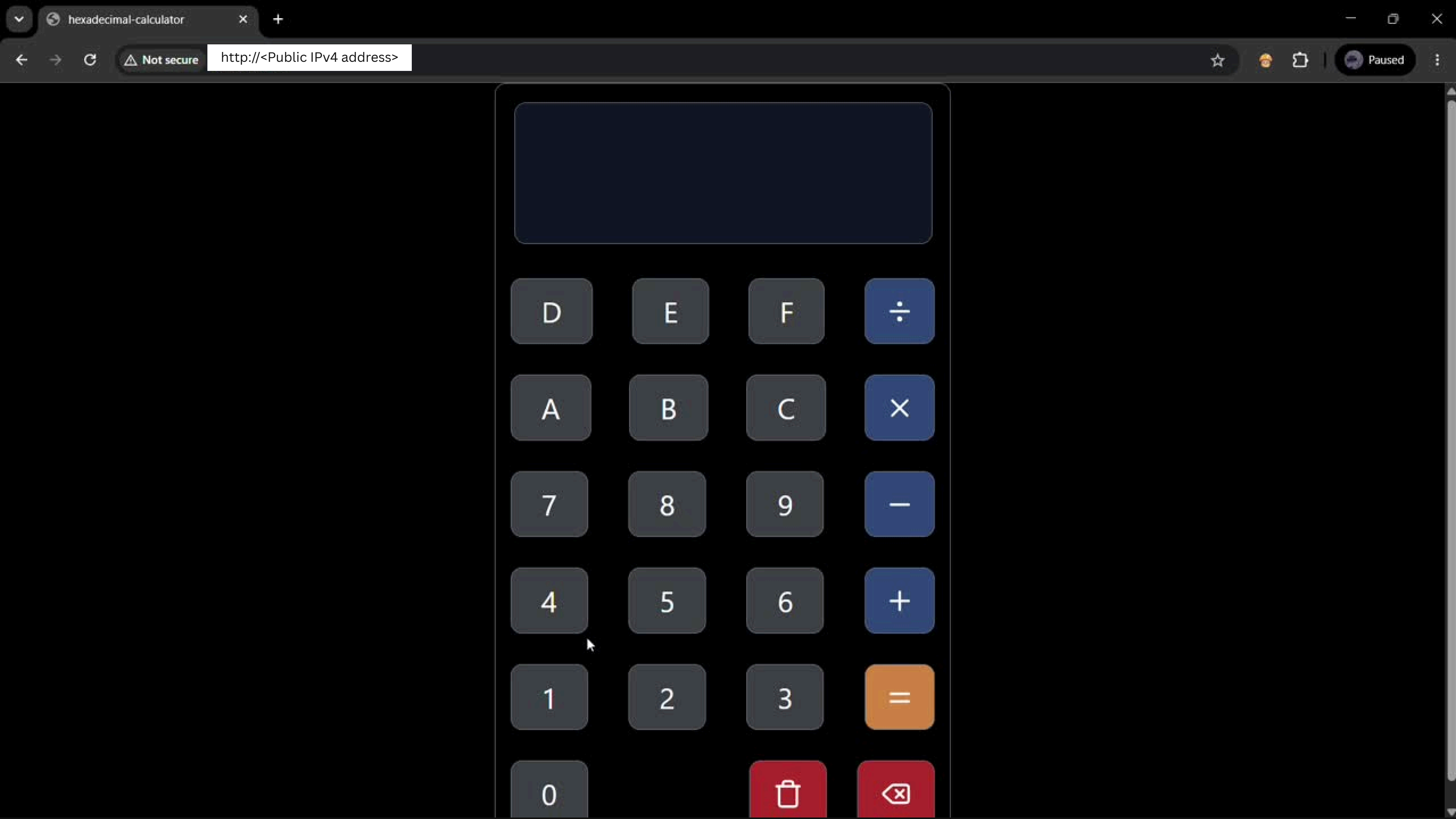
Tag a Release

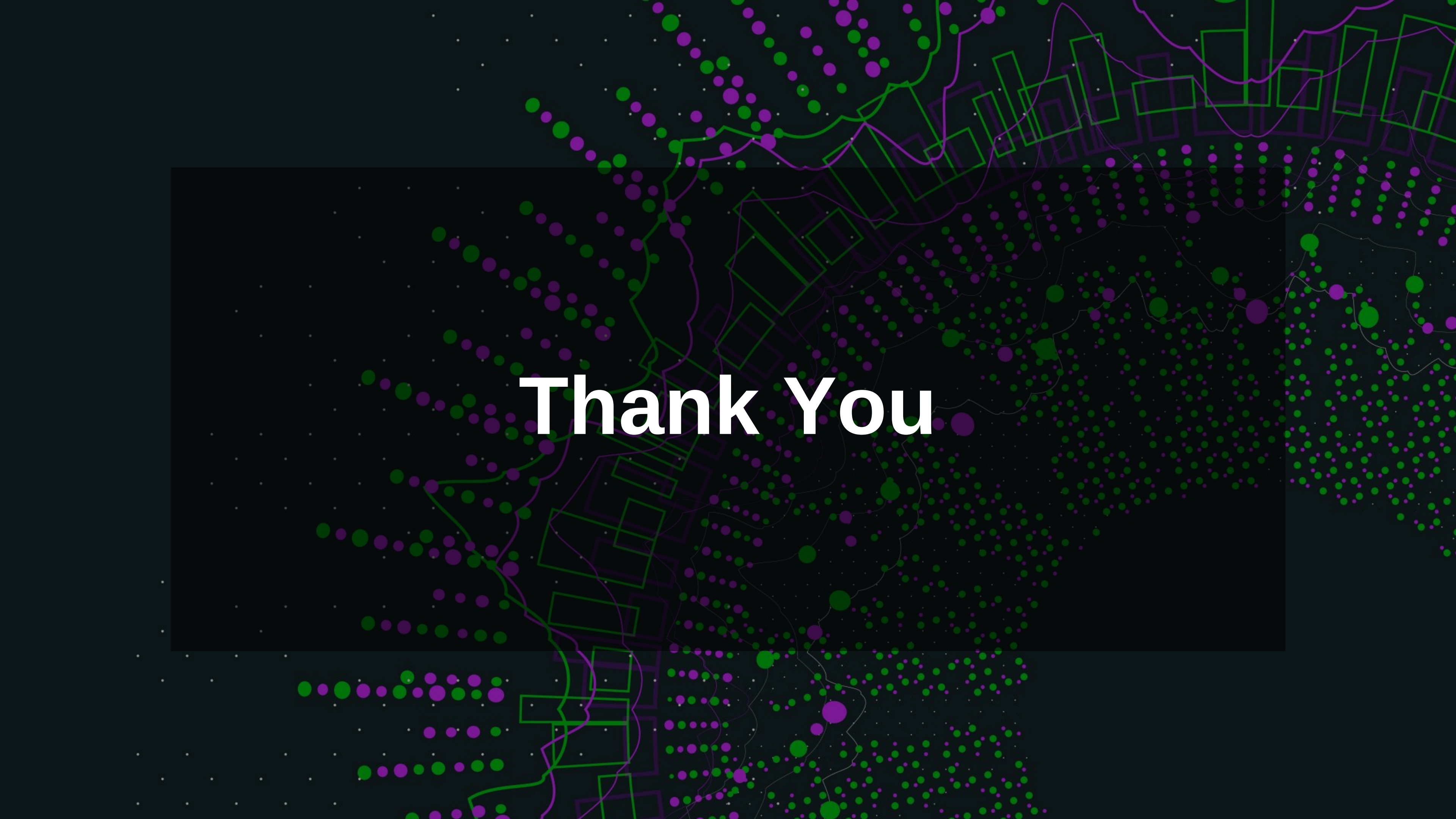
```
git tag v1.0.0  
git push origin v1.0.0
```



Verify Deployment







Thank You